

Appendix A

Introduction to Keras

Giovanni Della Lunga
giovanni.dellalunga@unibo.it

Advanced Machine Learning for Finance

Bologna - April-May, 2026

Keras is a popular open-source deep learning library that is written in Python. It is designed to be user-friendly, modular, and extensible, which makes it a great tool for beginners and experts alike. Here are some key points about Keras:

- User-friendly API: Keras provides a high-level API that makes it easy to build and train deep learning models. It supports both CPU and GPU computations, and it runs seamlessly on different platforms.
- Modular architecture: Keras is built on a modular architecture, which means that it provides a set of building blocks that you can combine to create complex deep learning models. This allows you to experiment with different architectures and see what works best for your problem.

- Starting from 2019, Keras does not supports multiple backends but it is strongly integrated into tensorflow.
- Pre-built models and layers: Keras provides a set of pre-built models and layers that you can use to jumpstart your deep learning projects. These models and layers are optimized for different tasks, such as image recognition, natural language processing, and time series analysis.
- Easy customization: Keras allows you to customize your models and layers easily. You can add or remove layers, change the hyperparameters, and fine-tune the models to improve their performance.

- Built-in utilities: Keras provides a set of built-in utilities that make it easy to preprocess your data, visualize your models, and monitor your training progress. These utilities can save you a lot of time and effort when working on deep learning projects.
- Large community: Keras has a large and active community of users and contributors, who share their knowledge and experience through forums, blogs, and tutorials. This means that you can find resources and help easily when you run into issues or have questions.
- Integration with other libraries: Keras integrates seamlessly with other Python libraries, such as NumPy, Pandas, and Scikit-learn. This allows you to use these libraries together with Keras to preprocess your data, analyze your results, and build end-to-end deep learning pipelines.

Introduction to keras: a practical problem

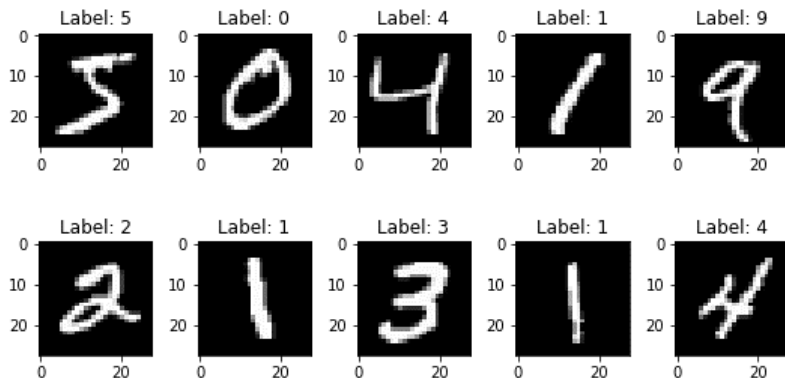
- The problem of automatic recognition of handwritten numbers, also known as digit recognition, is a classic problem in the field of pattern recognition and computer vision. It involves the task of automatically identifying and classifying handwritten digits into their corresponding numerical values.
- The challenge of digit recognition arises due to the variability in handwriting styles and the different ways in which people write the same digit. This variation can be caused by factors such as different writing instruments, writing speed, and writing pressure. Additionally, digits can be written in different sizes, orientations, and positions, further adding to the complexity of the problem.

Introduction to keras: a practical problem

- The problem we are trying to solve here is to classify grayscale images of handwritten digits (28 pixels by 28 pixels), into their 10 categories (0 to 9).
- The dataset we will use is the MNIST dataset, a classic dataset in the machine learning community, which has been around for almost as long as the field itself and has been very intensively studied.
- It's a set of 60,000 training images, plus 10,000 test images, assembled by the National Institute of Standards and Technology (the NIST in MNIST) in the 1980s.

Introduction to keras: a practical problem

- As we have said, in the MNIST dataset each digit is stored in a grayscale image with a size of 28x28 pixels.
- In the following you can see the first 10 digits from the training set:



Introduction to keras: Loading the dataset

- Keras provides seven different datasets, which can be loaded in using Keras directly.
- These include image datasets as well as a house price and a movie review datasets.
- The MNIST dataset comes pre-loaded in Keras, in the form of a set of four Numpy arrays

```
import keras

from keras.datasets import mnist

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()
```

The typical Keras workflows

- Define your training data: input tensor and target tensor
- Define a network of layers(or model) that maps input to our targets.
- Configure the learning process by choosing a loss function, an optimizer, and some metrics to monitor.
- Iterate your training data by calling the `fit()` method of your model.

Introduction to keras: Layers

- This is the building block of neural networks which are stacked or combined together to form a neural network model.
- It is a data-preprocessing module that takes one or more input tensors and outputs one or more tensors.
- These layers together contain the network's knowledge.
- Different layers are made for different tensor formats and data processing.

Creating a model with the sequential API

- The easiest way of creating a model in Keras is by using the sequential API, which lets you stack one layer after the other.
 - The problem with the sequential API is that it doesn't allow models to have multiple inputs or outputs, which are needed for some problems.
 - Nevertheless, the sequential API is a perfect choice for most problems.
-

```
from keras import models
from keras import layers

network = models.Sequential()
network.add(layers.Dense(512, activation='relu', input_shape=(28 * 28,)))
network.add(layers.Dense(10, activation='softmax'))
```

Introduction to keras

Let's go through this code line by line:

- The first line creates a Sequential model.
 - This is the simplest kind of Keras model, for neural networks that are just composed of a single stack of layers, connected sequentially.
 - This is called the **sequential** API.
-

```
from keras import models
from keras import layers

network = models.Sequential()
```

Introduction to keras

Next, we build the first layer and add it to the model.

- It is **Dense** hidden layer with 512 neurons.
- It will use the ReLU activation function.
- Each Dense layer manages its own weight matrix, containing all the connection weights between the neurons and their inputs.
- It also manages a vector of bias terms (one per neuron).
- When it receives some input data, it computes

$$\phi\left(Z^{[1]} = W^{[1]} \cdot X + B^{[1]}\right), \quad \phi(z) = \text{ReLU}(z)$$

```
network.add(layers.Dense(512, activation='relu', input_shape=(28 * 28,)))
```

Introduction to keras

Finally, we add a Dense output layer with 10 neurons (one per class).

- Using a 10-way "softmax" layer means that it will return an array of 10 probability scores (summing to 1).
- Each score will be the probability that the current digit image belongs to one of our 10 digit classes.

```
network.add(layers.Dense(10, activation='softmax'))
```

Introduction to keras

- The model's `summary()` method displays all the model's layers, including each layer's name (which is automatically generated unless you set it when creating the layer), its output shape (None means the batch size can be anything), and its number of parameters.

```
network.summary()
```

Introduction to keras

- The summary ends with the total number of parameters, including trainable and non-trainable parameters.
- Here we only have trainable parameters.

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 512)	401920
dense_3 (Dense)	(None, 10)	5130
Total params: 407,050		
Trainable params: 407,050		
Non-trainable params: 0		

Compile a Model

- Before we can start training our model we need to configure the learning process.
- For this, we need to specify an optimizer, a loss function and optionally some metrics like accuracy.
- The **loss function** is a measure on how good our model is at achieving the given objective.
- An **optimizer** is used to minimize the loss(objective) function by updating the weights using the gradients.

Loss Function

- Choosing the right Loss Function for the problem is very important, the neural network can take any shortcut to minimize the loss.
- So, if the objective doesn't fully correlate with success for the task at hand, your network will end up doing things you may not have wanted.

Loss Function

For common problems like Classification, Regression and Sequence prediction, they are simple guidelines to choose a loss function.

For:

- Two- Class classification you can choose binary cross-entropy
- Multi-Class Classification you can choose Categorical Cross-entropy.
- Regression Problem you can choose Mean-Squared Error

Activation Function

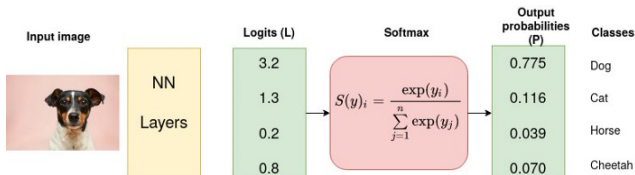
Softmax Activation Function

- Softmax is an activation function that scales numbers/logits into probabilities.
- The output of a Softmax is a vector (say v) with probabilities of each possible outcome.
- The probabilities in vector v sums to one for all possible outcomes or classes
- It is often used as the last activation function of a neural network to normalize the output of a network to a probability distribution over predicted output classes
- You can think of softmax function as a sort of generalization to multiple dimension of the logistic function

Activation Function

Softmax Activation Function. Mathematically, softmax is defined as

$$S(y)_i = \frac{\exp y_i}{\sum_{j=1}^n \exp y_j} \quad (1)$$



Loss Function

Categorical Cross-Entropy

- Remember the logistic Loss Function:

$$L(y, \hat{y}) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y}) = - \sum_{j=1}^2 y_j \log \hat{y}_j$$

- Generalizing this result to the n-class classification problem we have

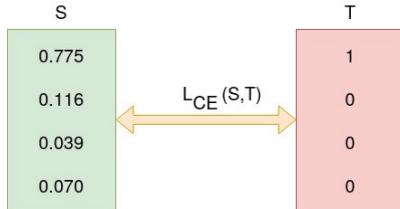
$$L = - \sum_{j=1}^{N_C} y_j \log \hat{y}_j \quad (2)$$

This last equation define the so called *cross-entropy*

Loss Function

Categorical Cross-Entropy

- In the previous example, Softmax converts logits into probabilities.
- The purpose of the Cross-Entropy is to take the output probabilities (P) and measure the distance from the truth values



Categorical Cross-Entropy

- The categorical cross-entropy is computed as follows

$$\begin{aligned} L_{CE} &= - \sum_{i=1} T_i \log(S_i) \\ &= - [1 \log_2(0.775) + 0 \log_2(0.126) + 0 \log_2(0.039) + 0 \log_2(0.070)] \\ &= - \log_2(0.775) \\ &= 0.3677 \end{aligned}$$

RMSProp

- Gradient descent is an optimization algorithm that follows the negative gradient of an objective function in order to locate the minimum of the function.
- A limitation of gradient descent is that it uses the same step size (learning rate) for each input variable.
- AdaGrad is an extension of the gradient descent optimization algorithm that allows the step size in each dimension used by the optimization algorithm to be automatically adapted based on the gradients seen for the variable (partial derivatives) over the course of the search.
- A limitation of AdaGrad is that it can result in a very small step size for each parameter by the end of the search that can slow the progress of the search down too much and may mean not locating the optima.
- Root Mean Squared Propagation, or RMSProp, is an extension of gradient descent and the AdaGrad version of gradient descent that uses a decaying average of partial gradients in the adaptation of the step size for each parameter.
- The use of a decaying moving average allows the algorithm to forget early gradients and focus on the most recently observed partial gradients seen during the progress of the search, overcoming the limitation of AdaGrad.

Compile the model

So, to make our network ready for training, we need to pick three things, as part of "compilation" step:

- A loss function: this is how the network will be able to measure how good a job it is doing on its training data, and thus how it will be able to steer itself in the right direction.
- An optimizer: this is the mechanism through which the network will update itself based on the data it sees and its loss function.
- Metrics to monitor during training and testing. Here we will only care about accuracy (the fraction of the images that were correctly classified).

```
network.compile(optimizer='rmsprop',  
                loss='categorical_crossentropy',  
                metrics=['accuracy'])
```

Introduction to keras: Training

- Before training, we will preprocess our data by reshaping it into the shape that the network expects, and scaling it so that all values are in the '[0, 1]' interval.
- Previously, our training images for instance were stored in an array of shape '(60000, 28, 28)' of type 'uint8' with values in the '[0, 255]' interval.
- We transform it into a 'float32' array of shape '(60000, 28 * 28)' with values between 0 and 1.

```
train_images = train_images.reshape((60000, 28 * 28))  
train_images = train_images.astype('float32') / 255
```

```
test_images = test_images.reshape((10000, 28 * 28))  
test_images = test_images.astype('float32') / 255
```

Introduction to keras: Training

We also need to categorically encode the labels:

```
from keras.utils.np_utils import to_categorical  
  
train_labels = to_categorical(train_labels)  
test_labels = to_categorical(test_labels)
```

Introduction to keras: Training

What is an epoch?

- An epoch is a term used in machine learning and indicates the number of passes of the entire training dataset the machine learning algorithm has completed. Datasets are usually grouped into batches (especially when the amount of data is very large). Some people use the term iteration loosely and refer to putting one batch through the model as an iteration.
- If the batch size is the whole training dataset then the number of epochs is the number of iterations. For practical reasons, this is usually not the case. Many models are created with more than one epoch. The general relation where dataset size is d , number of epochs is e , number of iterations is i , and batch size is b would be $d \cdot e = i \cdot b$.
- Determining how many epochs a model should run to train is based on many parameters related to both the data itself and the goal of the model, and while there have been efforts to turn this process into an algorithm, often a deep understanding of the data itself is indispensable.

Introduction to keras: Training

We are now ready to train our network, which in Keras is done via a call to the 'fit' method of the network: we "fit" the model to its training data.

```
history = network.fit(train_images, train_labels, epochs=5, batch_size=128)
```

Visualizing the training process

- We can visualize our training and testing accuracy and loss for each epoch so we can get intuition about the performance of our model.
 - The accuracy and loss over epochs are saved in the history variable we got whilst training and we will use Matplotlib to visualize this data.
-

```
# list all data in history
print(history.history.keys())
# summarize history for accuracy
plt.plot(history.history['mae'])
plt.plot(history.history['val_mae'])
plt.title('model mae')
plt.ylabel('mae')
plt.xlabel('epoch')
plt.legend(['train', 'test'], loc='upper left')
plt.show()
```
